DEVOPS OVERVIEW

Nguyễn Thanh Quân - ntquan@fit.hcmus.edu.vn

Agenda

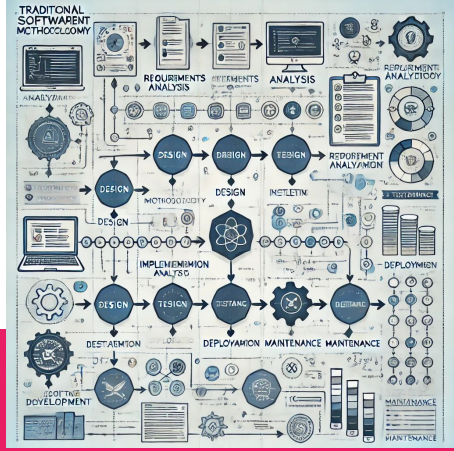
---

1. Traditional Software Development Methodology
2. Introduction to DevOps
3. Addressing Challenges through DevOps
4. Agile Methodology
5. The Relationship between Agile and DevOps
6. DevOps Toolchain
7. DevOps Principles
8. DevOps Best Practices

— — —

Terminology	Explanation
Development env	An environment used for developing and testing new features at the individual or small group level.
Staging env	Môi trường mô phỏng production để kiểm thử toàn diện trước khi triển khai thực tế.
Production env	A realistic operating environment, serving end users and requiring the highest level of stability.
PoC (Proof of Concept)	It's a small-scale test to check if a solution or technology is feasible before investing in building a full system.

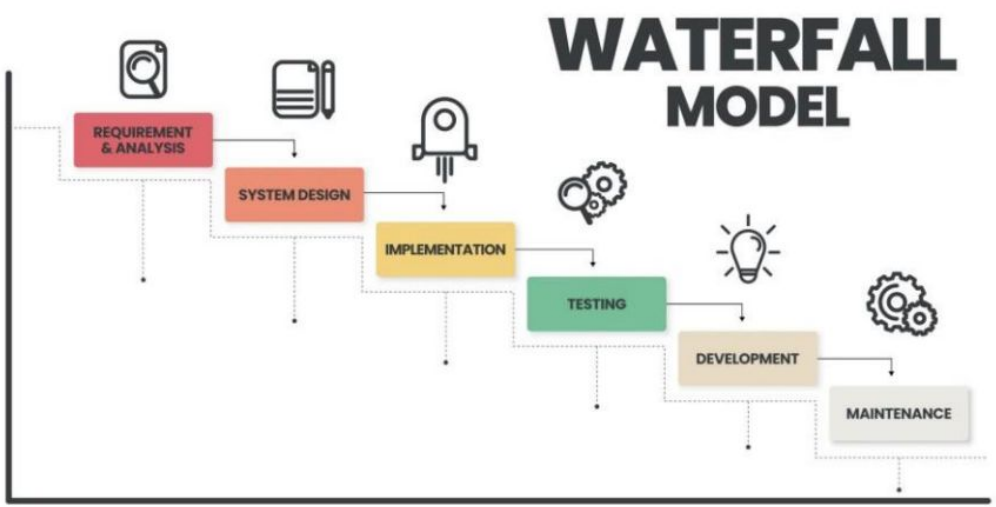
Traditional Software Development Methodology



Traditional Software Development Methodology

Waterfall Methodology:

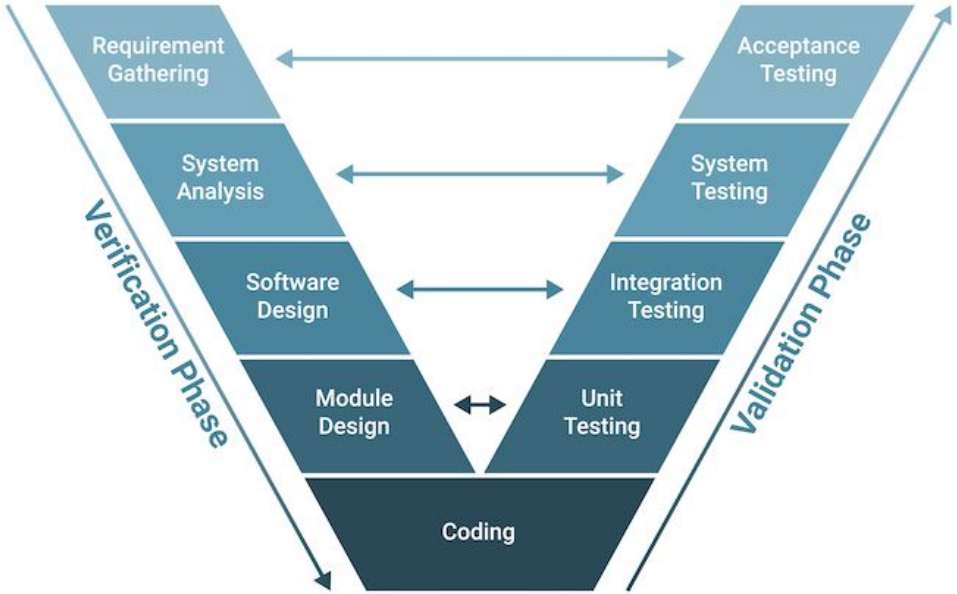
The Waterfall model is a linear sequential software development methodology in which development stages (planning, requirements analysis, design, implementation, testing) are carried out in distinct steps, and there is no way to return to previous steps once a stage is completed.



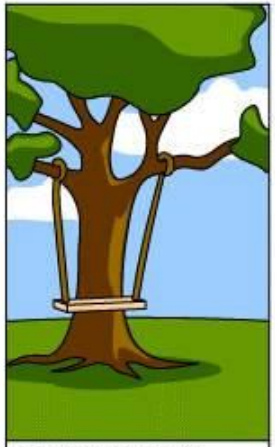
Traditional Software Development Methodology

V-Model Methodology:

The V-Model is a software development methodology similar to the Waterfall model but integrates development and testing phases. It requires testing to be planned and executed from the very beginning of the software development process.

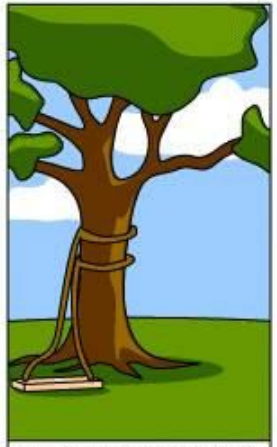


How the customer explained it



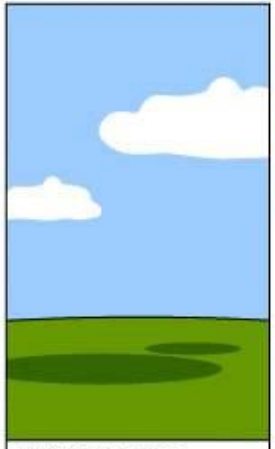
How the Project Leader understood it

How the Analyst designed it

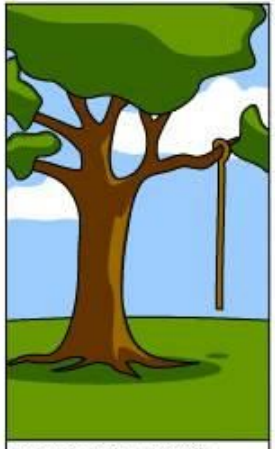


How the Programmer wrote it

How the Business Consultant described it

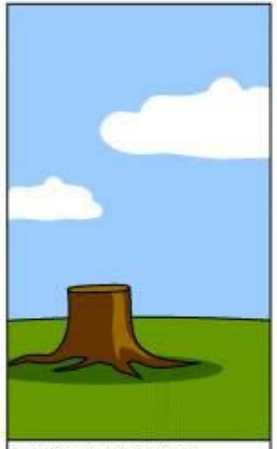


How the project was documented



What operations installed

How the customer was billed



How it was supported

What the customer really needed

Traditional Software Development Methodology

Challenges with Traditional Approaches

1. Time and Schedule Challenges

Lengthy and Unpredictable Timeframes: Traditional methodologies often involve long project durations with rigid phase transitions, making it difficult to adapt to changing requirements or new technologies.

Risk of Missing Requirements: Overly detailed planning at the beginning, without early feedback, can result in developing the wrong solutions or failing to meet user needs effectively.

Traditional Software Development Methodology

Challenges with Traditional Approaches

2. Challenges in Managing Changes

Difficulty in Changing Requirements: The Waterfall methodology makes it difficult to accommodate changes during the development process, leading to costly and complex adjustments in later stages of the project.

Errors Arising from Changes: very requirement change affects parts of the system that have already been developed, increasing the likelihood of introducing errors into the software.

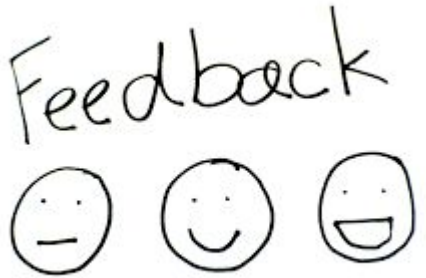
Traditional Software Development Methodology

Challenges with Traditional Approaches

3. Lack of Quick Feedback

Delayed Feedback During Development: Traditional software development teams typically receive feedback from customers only after the software is fully deployed. This can result in the final product failing to meet user needs effectively.

Absence of Continuous Collaboration: This approach does not encourage ongoing collaboration between development teams and customers, making it difficult to implement changes or optimize the product during the development process.



Traditional Software Development Methodology

Challenges with Traditional Approaches

4. Challenges in Testing and Deployment

Late Testing: Testing is typically performed at the end of the process (after the code has been fully developed), which can lead to late detection of errors, increasing the cost and effort required to fix them.

Difficult Deployment: Deploying software into a real-world environment in traditional methodologies is often challenging, as there is no test environment that mirrors the production environment, increasing deployment risks.

Traditional Software Development Methodology

Challenges with Traditional Approaches

5. Waste and Inefficiency

Resource Wastage: Stages like requirements analysis or design may need to be redone multiple times, resulting in wasted time and resources.

Difficulty in Achieving Flexibility: The linear nature of traditional methodologies requires teams to redo previous steps before moving forward, leading to time delays, especially when requirements change.

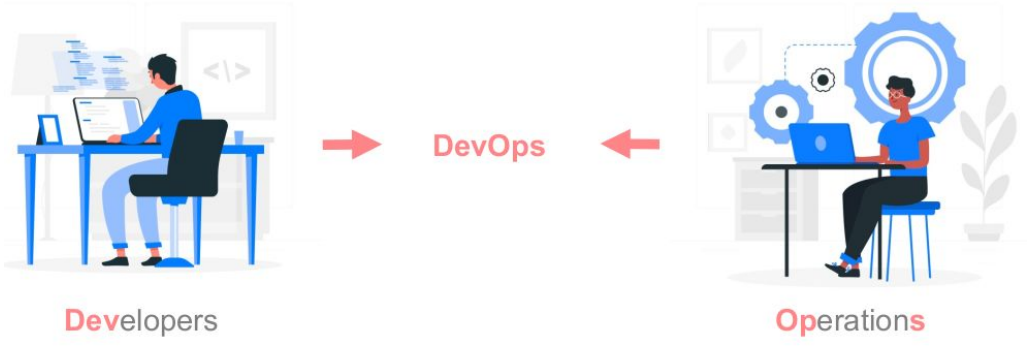
Introduction to DevOps

What is Devops

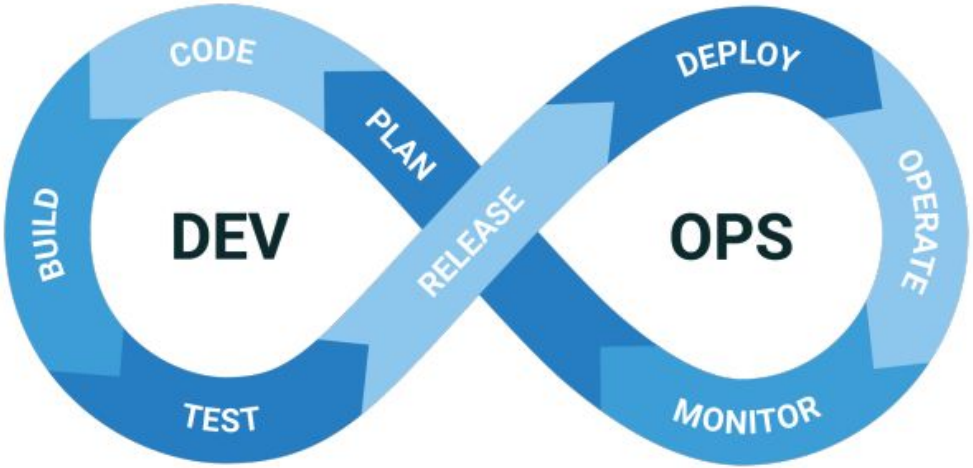
- DevOps is a working methodology applied in the IT software development industry.

- DevOps combines practices, a collaborative working culture, and tools that enhance an organization's ability to deliver applications and services at a high level.

- The name DevOps comes from Developer and Operations. DevOps bridges the communication gap between the software developers and the IT operation teams.



Introduction to DevOps



Devops Lifecycle

Introduction to DevOps

---

1. Plan - This stage involves initial planning for how you envision the development process, including defining project goals and requirements.
2. Code - Writing the code for applications or features according to the organization's or company's requirements.
3. Build - Integrating the written code into a build process, ensuring everything works together properly.
4. Test - Testing the application to ensure that it is ready
5. Releases - If the testing phase is successful, the application is ready to be deployed and made operational.

Introduction to DevOps

---

6. Deploy - The code is deployed to the production environment.

7. Operate - The application is operated after deployment, ensuring it runs as expected.

8. Monitor - The application is continuously monitored, and necessary changes are made to ensure smooth and stable operation.

Addressing Challenges through DevOps

Addressing Challenges through DevOps

---

1. Enhancing Development and Deployment Speed

Challenge: Slow software development, unable to quickly respond to changing requirements.

Solution:

- Continuous Integration (CI) and Continuous Delivery - Deployment (CD): CI/CD automates the process of integrating source code and deploying to testing or production environments, reducing the time from development completion to product release.
- Automated Testing: Automated testing helps quickly identify errors during the development process, speeding up development and ensuring quality.

Addressing Challenges through DevOps

---

2. Improving Software Quality

Challenge: Software bugs and inconsistencies between the development environment and the production environment.

Solution:

- Automated Testing: DevOps promotes automating testing to catch errors early, from unit testing to integration testing and user acceptance testing.
- Consistent Environments: DevOps ensures that the development and production environments are similar, minimizing errors during deployment.

Addressing Challenges through DevOps

---

3. Managing Changes and Deployment Errors

Challenge: Managing changes to source code, new features, or deployment environments is difficult.

Solution:

- Configuration Management: DevOps automates infrastructure and software configuration management using tools like Ansible, Puppet, or Chef.
- Feature Toggles: Feature toggles allow for changes or testing of features without modifying the main source code, enabling rapid deployment without disruption.

Addressing Challenges through DevOps

---

4. Improving Collaboration Between Dev and Ops Teams

Challenge: Lack of collaboration between development and operations teams leads to deployment and maintenance issues.

Solution:

- Enhancing Collaboration and Communication: DevOps promotes connecting development, testing, and operations teams, helping reduce barriers and improve work efficiency.
- Task Management through Tools: Tools like Slack, JIRA, and Confluence facilitate easier and more effective collaboration, improving communication and progress tracking.

CAMS model

CAM model

CAMS = Culture + Automation + Measurement + Sharing

The CAMS model was proposed
by John Willis and Damon Edwards
as a foundational framework
for understanding the key components
in implementing the DevOps methodology

CAMS mode - Culture

Culture: Emphasizes creating a collaborative environment among development teams (Development), operations teams (Operations), and othe stakeholders.

Key elements:

- Breaking down the "barriers" betwee Development and Operations.
- Encouraging seamless continuous collaboration.
- Fostering a culture of shared responsibility.

CAMS mode - Automation

Automation: A key element in DevOps aimed at minimizing manual work, increasing efficiency, and ensuring consistency across processes.

Automate everything: From building, testing, deploying, to infrastructure management.

Benefits:

- Minimizes human errors.
- Accelerates software release cycles.
- Enhances scalability and consistency.

CAMS mode - Measurement

Measurement: Helps monitor and evaluate the performance of systems and processes.

Collect metrics and logs: Track elements such as response time, system performance, deployment failure rates, and application stability.

Importance of measurement:

- Early detection of issues in the product.
- Assessing system performance.

CAMS mode - Sharing

Sharing: DevOps promotes a culture of sharing knowledge, information, and case studies among teams and stakeholders.

What to share: Documentation, incident reports, performance reports, and measurement results to create a transparent and trustworthy ecosystem.

Key practices:

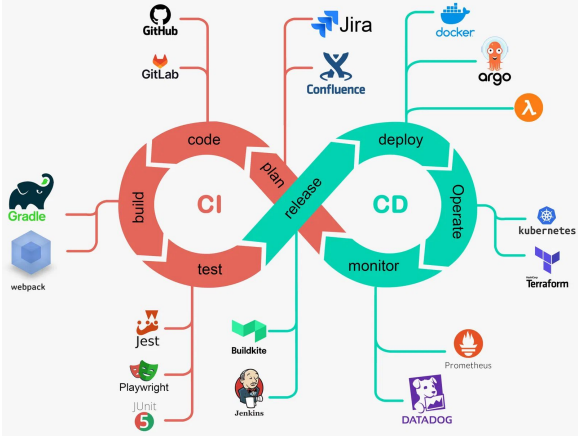
- Share information about errors and incidents to learn from failures.
- Organize review sessions to learn from past deployments.

CI/CD

CI/CD

CI = Continuous Integration

CD = Continuous Delivery or Continuous Deployment



These are two key methodologies in modern software development, enabling automation and improvement of the processes for software development, testing, and deployment.

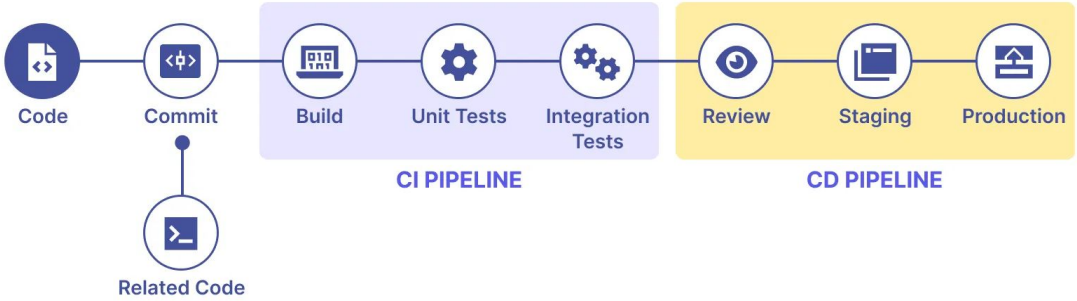
CI/CD is the foundation of DevOps → Ensures speed, quality, and continuity in software delivery.

CI - Continuous Integration

CI (Continuous Integration): A process where developers frequently commit their code to a shared source control repository.

Goals of CI:

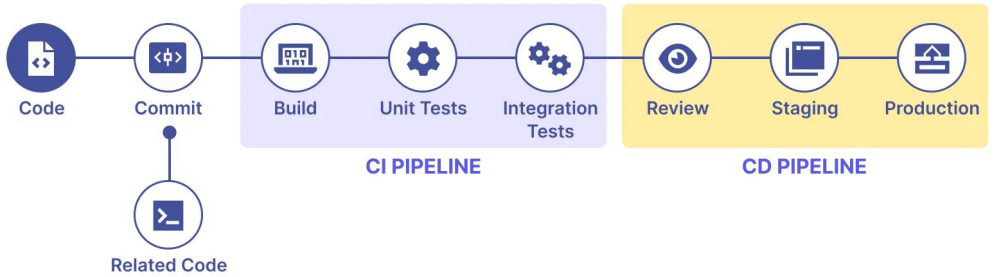
- Early error detection: By integrating and testing code, issues like bugs and conflicts between team members' code are identified early.
- Automation of build and test processes: Each time code changes, the CI system automatically runs tests to ensure that changes do not compromise application stability.



CI - Continuous Integration

Basic CI Process:

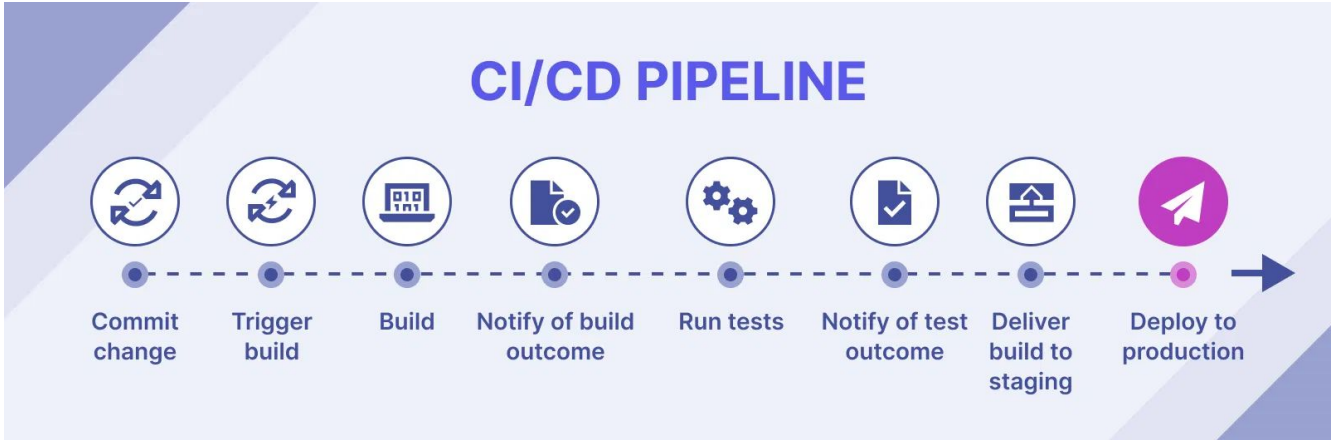
- Developers write code and commit it to a main branch (or development branch) in Git or another version control system.
- The CI system (e.g., Jenkins, CircleCI, GitLab CI) automatically builds the code and runs unit tests.
- If the build and tests succeed, the code is merged into the main branch and prepared for subsequent steps.



CD - Continuous Delivery

CD (Continuous Delivery): The next step after CI, focusing on automating the software release process.

Goal: Ensure that the integrated code is always ready to be deployed to the production environment after passing all tests.



CD - Continuous Delivery

Basic CD Process:

- After the code is continuously integrated (CI) and passes all tests, it is transferred to staging or testing environments.
- From here, the developer can choose to release the production build with minimal risk.

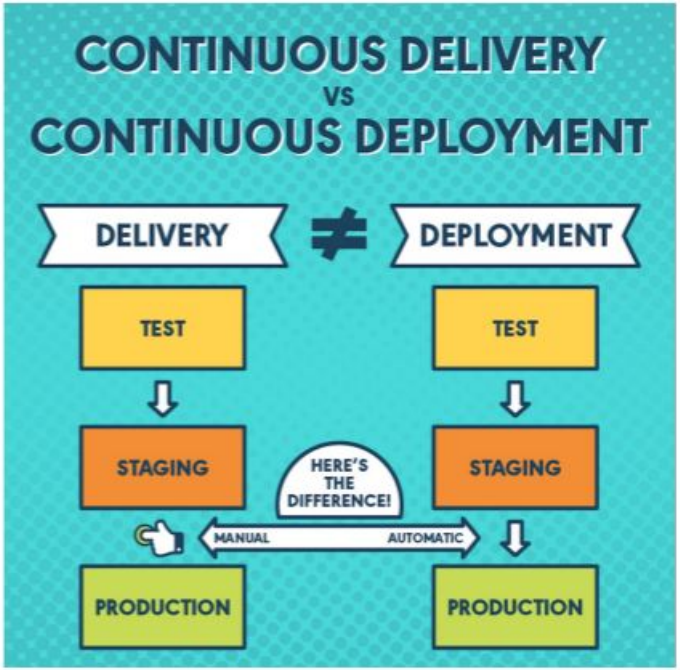
CD - Continuous Deployment

Continuous Deployment: A higher level of Continuous Delivery. Changes that have passed all tests are automatically deployed to the production environment without manual intervention.

CD

Comparison with Continuous Delivery:

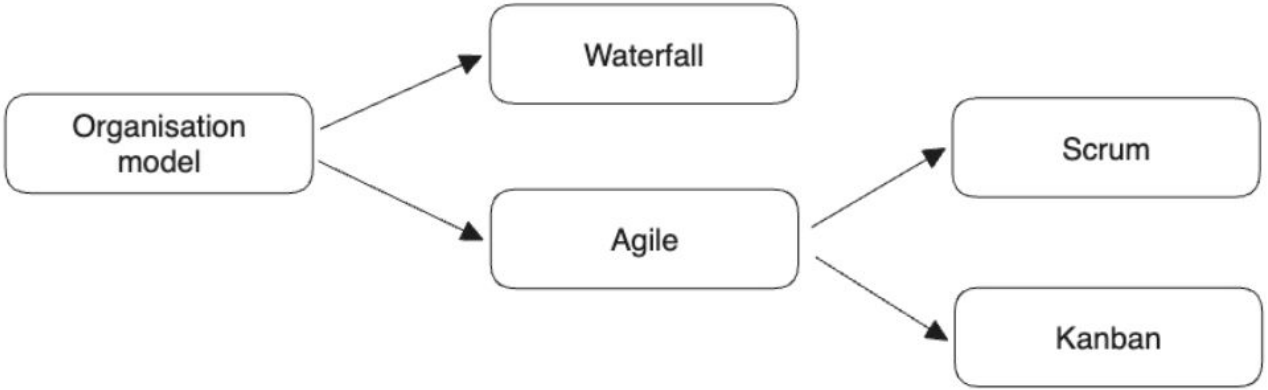
- Continuous Delivery stops at the preparation stage for deployment, and releasing to the production environment still requires manual approval.
- Continuous Deployment automatically deploys all changes to production, with no manual intervention required after tests are completed.



Agile Methodology

It’s hard to organise an IT project...

Different models for organising an IT project



AGILE

Agile is a flexible software development methodology that emphasizes collaboration, delivering products early and frequently, while also responding quickly to changes.

Waterfall vs Agile

Criteria	Agile	Waterfall
Approach	Iterative and Incremental	Sequential
Flexibility	Highly flexible and adaptable	Less flexible
Planning	~~Minimal~~ Less planning is enough	Detailed planning required
Customer Involvement	High level of customer involvement	Low level of customer involvement
Risk management	Continuous risk management	Risk management at the beginning of the project
Documentation	~~Minimal~~ Less documentation required	Extensive documentation required
Time and cost	Challenging to estimate time and cost	Easy to estimate time and cost

https://testsigma.com/blog/difference-between-agile-and-waterfall/

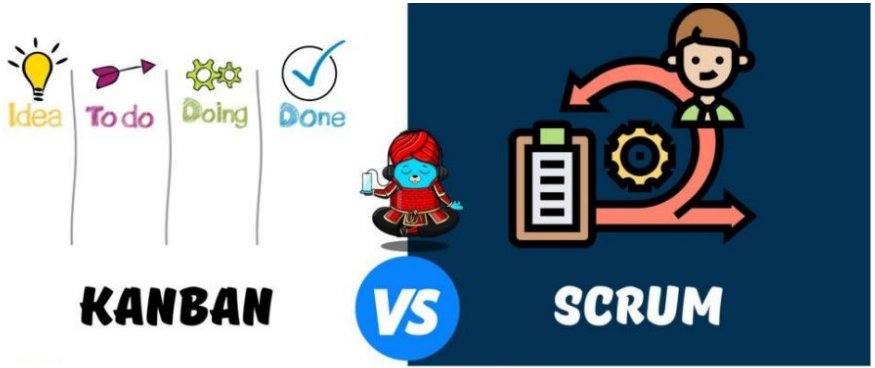
AGILE

---

4 Core Values:

1. Individuals and Interactions over Processes and Tools
2. Working Software over Comprehensive Documentation
3. Collaboration with Customers over Contract Negotiation
4. Responding to Change over Following a Plan

AGILE



Framework Agile

1. Scrum

- Roles: Product Owner, Scrum Master, Development Team.
- Events: Sprint Planning, Daily Standup, Sprint Review, Sprint Retrospective.
- Artifacts: Product Backlog, Sprint Backlog, Increment.

2. Kanban

- Principles: Visualize Work, Limit Work in Progress (WIP), Continuous Improvement
- Tools: Kanban Board

...

Kanban methodology

In Japanese, kanban literally translates to "visual signal."

- Originally comes from lean manufacturing in Japan (at Toyota)
- Simple and clean application of Agile
- Matches the amount of Work in Progress (WIP) to the team’s capacity - Just In Time (JIT)
- Minimal set of rules - easy to pick up for new software engineering teams

Advantages

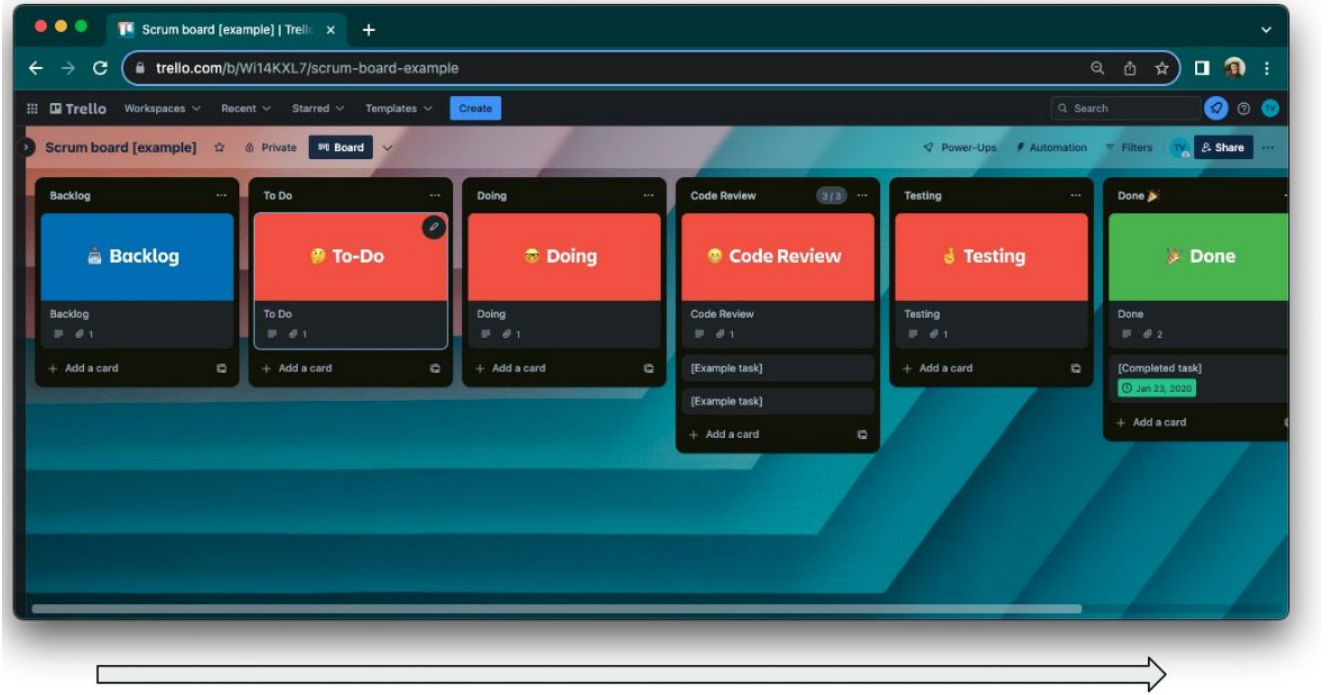
- Planning flexibility
  - Once done with an item, pick up the next on the backlog
  - Product owner can update the backlog without disrupting the team
- Short time cycles
  - Time for a unit of work to cycle through the whole process
- Fewer bottlenecks
- Visual metrics, transparency

Kanban board

The **kanban board** is filled with **kanban cards** (aka tickets or tasks).

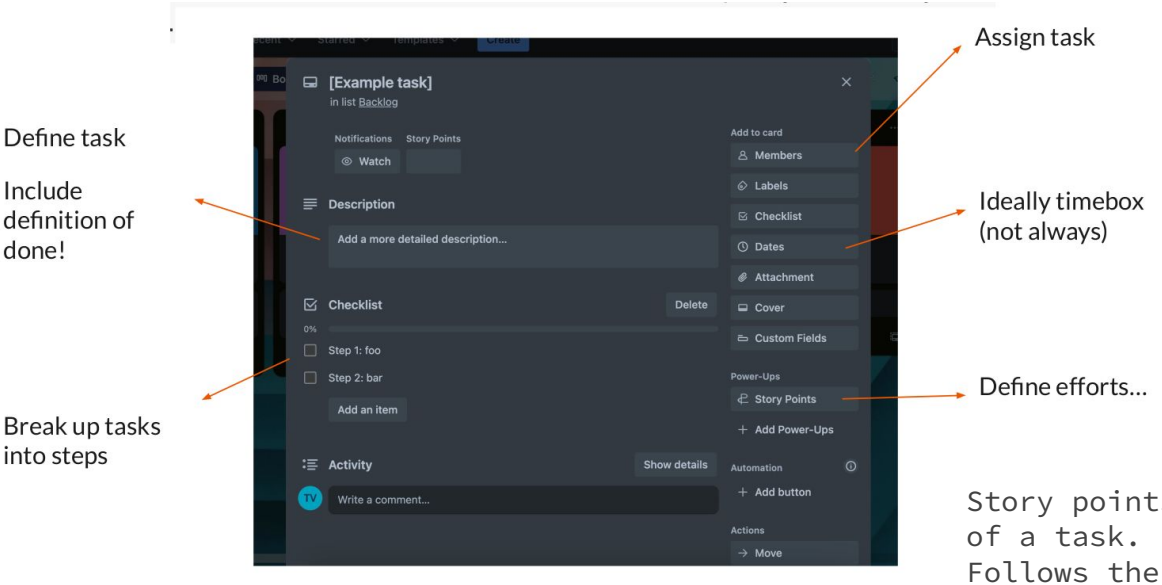
New tasks are added to the **backlog.**

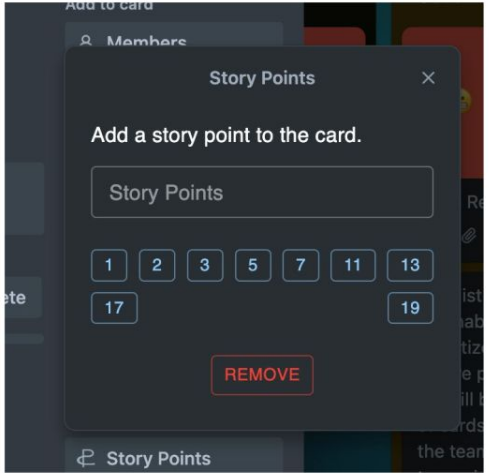
They are then gradually **moved** along the board.



Kanban card

A detailed description of what should be done and how ensures quality of delivery and alignment in the team.

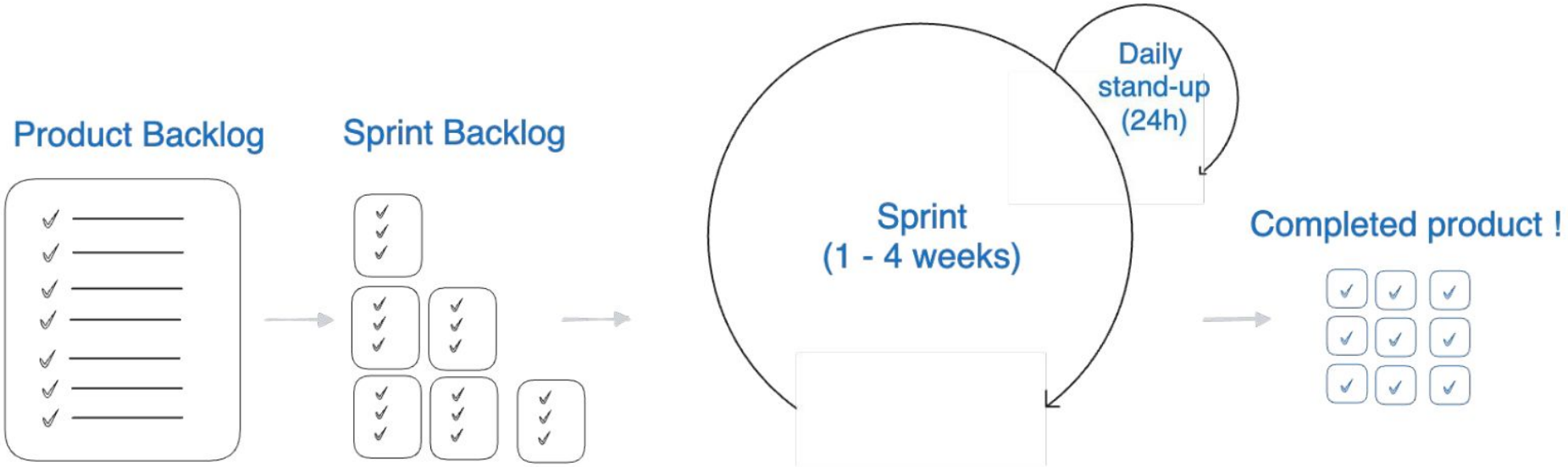




more information:
https://www.youtube.com/watch?v=CD0y-aU1sXo

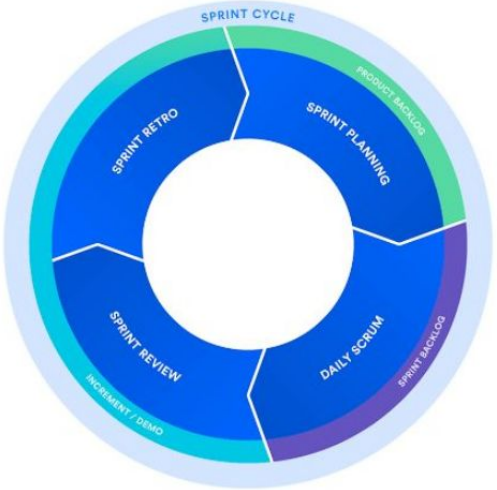
Story points are a measure of effort and complexity of a task.
Follows the fibonacci scale.
Sometimes voted on (e.g. https://www.pointingpoker.com/)

Scrum methodology



Scrum methodology

- Scrum is organised around 1-4 weeks sprint cycles (often 2 weeks).
- **Sprint planning**: Before each sprint, plan and add tasks from the product backlog to the sprint backlog.
- **Daily stand-up**: (Aka daily scrum) During the sprint, have dailies to update the scrum board, similar to kanban board. Go over each team member’s progress.
- **Sprint review**: Casually present the work that was done in the last sprint to the rest of the team (definition, celebration and transparency).
- **Sprint retrospective**: Feedback on what went well and what can be improved regarding last sprint.



Sprint organisation - Daily stand-up

Goal is to make the meeting efficient.

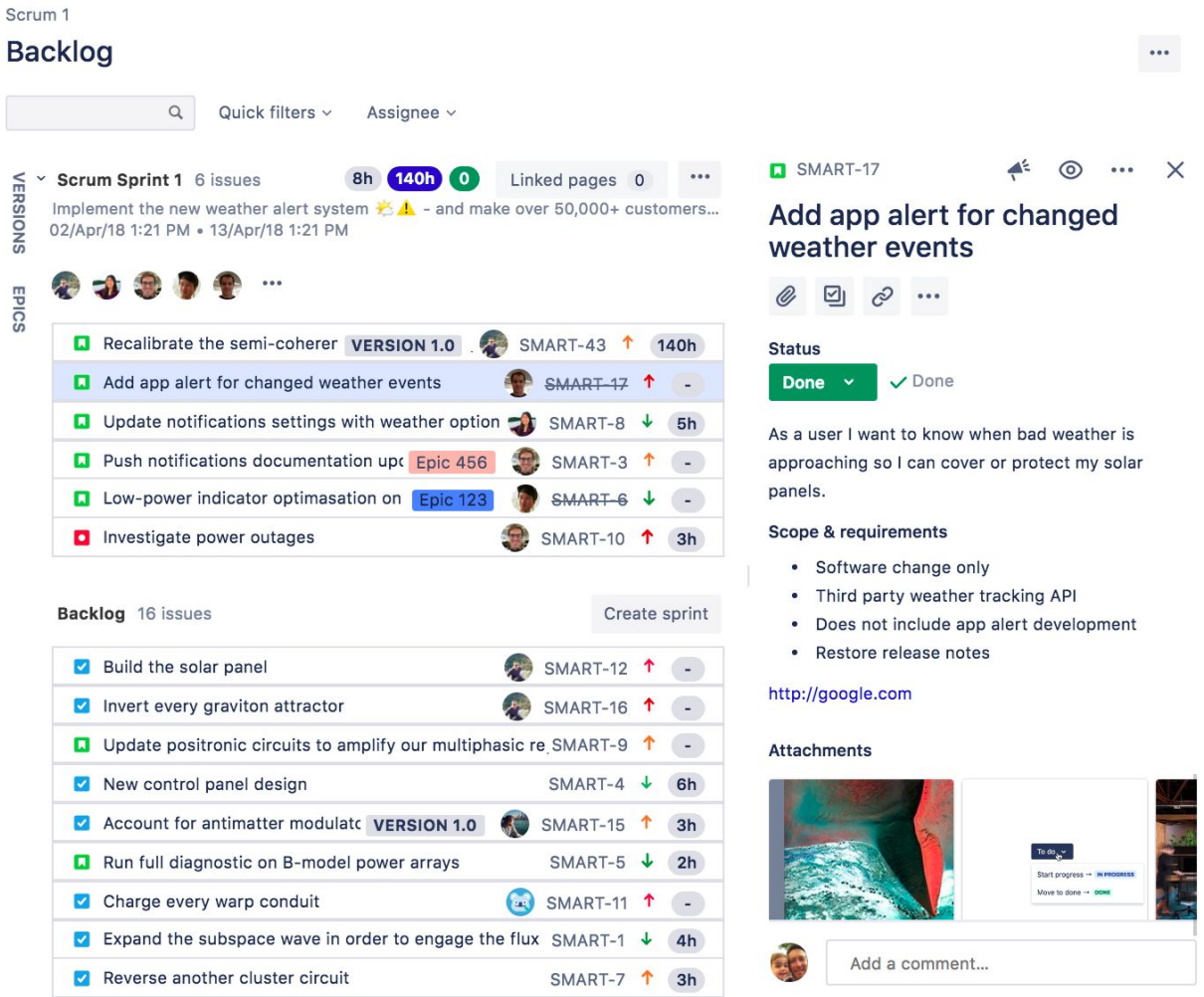
All team members have to answer a fixed set of questions such as:

- What did you do yesterday?
- What did you do today?
- What, if anything, is blocking your progress?

Scrum - The scrum team

Roles & responsibilities

- **Product Owner**: Represents stakeholders' interests. Selects what is relevant to work on considering product objectives. Define user stories, priorities backlog and decide on product's direction.
- **Scrum Master**: Coach for the team. Ensures the best following of the scrum framework. Maintains the scrum board and backlog, facilitates and leads meetings and addresses obstacles.
- **Scrum Development Team**: Cross-functional developer team (data scientists, data engineers, DevOps, front-end engineers, back-end engineers, …). Delivers a quality product.



https://confluence.atlassian.com

AGILE

Benefits of Agile:

- **Reduced Product Development Time**: Agile practices, such as iterative development, allow for faster delivery of features and product releases.
- **Increased Customer Satisfaction**: Continuous feedback from customers and regular product updates ensure that the product meets their needs and expectations.
- **Enhanced Flexibility and Adaptability**: Agile allows teams to easily adapt to changes in requirements, technology, or market conditions, ensuring the product remains relevant and aligned with business goals.

The Relationship between Agile and DevOps

Agile and DevOps

---

1. Goals: Both Agile and DevOps aim to improve speed, quality, and responsiveness in software development:

- Agile: Focuses on accelerating development through flexible processes, allowing for quick adjustments and regular delivery of value.
- DevOps: Focuses on optimizing deployment and operations of software by automating and continuously integrating processes, ensuring efficient and reliable delivery to production.

Agile and DevOps

---

2. Agile Prepares the Foundation for DevOps

- Agile Manages the Development Process: Agile creates small, deployable products or increments after each sprint, which aligns well with DevOps, where these small increments are continuously tested and deployed through Continuous Integration (CI) and Continuous Deployment (CD).
- DevOps Deploys Agile Products: After the product is developed using Agile methodologies, DevOps ensures quick and stable deployment through automation and containerization, making the process seamless and reliable.

Agile and DevOps

---

3. Collaboration in Software Development Stages

Stage	Role of Agile	Role of DevOps
Planning	Flexible planning according to sprints, managing the backlog.	Ensures a consistent environment for deployment.
Development	Develop small increments, frequent communication.	Automates source code testing (CI).
Testing	Continuous feedback from customers and QA teams.	Automates testing and integrates with the pipeline.
Deployment	Not directly involved in deployment.	Automatically deploys source code to production environments.
Operations & Maintenance	Improves software through customer feedback.	Monitors and tracks system performance (monitoring).

Agile and DevOps

---

4. Culture and Collaboration

Both Agile and DevOps emphasize high levels of collaboration between teams:

- Agile: Focuses on communication and interaction among development team members. It encourages regular collaboration through daily stand-ups, sprint reviews, and feedback loops, fostering teamwork and flexibility.
- DevOps: Bridges the gap between development (Development) and operations (Operations) teams to create a seamless working culture. This integration fosters collaboration, continuous feedback, and efficient processes for deploying and maintaining software in production.

Agile and DevOps

---

5. Continuous Improvement Capability

Both Agile and DevOps emphasize high levels of collaboration between teams:

- Agile: Improves the product through Sprint Reviews and Retrospectives.
- DevOps: Improves system performance through measurement, monitoring, and automation.

Devops Toolchain

Devops Toolchain

DevOps Toolchain: A set of integrated tools used to automate the entire software development lifecycle (from coding, testing, deployment, to monitoring).

The DevOps Toolchain supports DevOps principles such as automation, continuous integration (CI), continuous delivery (CD), and system monitoring.

Devops Toolchain

Tools in the DevOps Toolchain:

1. Version Control: Tools like Git, Subversion (SVN), and Mercurial to manage code versions and collaborate among team members.

Devops Toolchain

Tools in the DevOps Toolchain:

2. Containerization and Orchestration Tools

- Docker: A containerization tool that allows you to package and deploy software in isolated environments, known as containers.
- Kubernetes: An orchestration platform that helps manage and automate the deployment, scaling, and operation of containerized applications.

kubernetes

Devops Toolchain

Tools in the DevOps Toolchain:

3. Automated Testing Tools

- Selenium: A widely-used tool for automating web applications across different browsers. It supports multiple programming languages like Java, Python, and C# for writing test scripts.
- JUnit/TestNG: A framework used for unit testing in Java applications.

Devops Toolchain

Tools in the DevOps Toolchain:

4. Configuration Management Tools

- Ansible: A simple, agentless automation tool for managing configurations, applications, and infrastructure. It uses YAML for writing playbooks and can manage both Linux and Windows systems.
- Chef, Puppet: Configuration management tools similar to Ansible, but with different approaches:
    - Chef: Uses a Domain-Specific Language (DSL)
    - Puppet: Uses a declarative language

Devops Toolchain

Tools in the DevOps Toolchain:

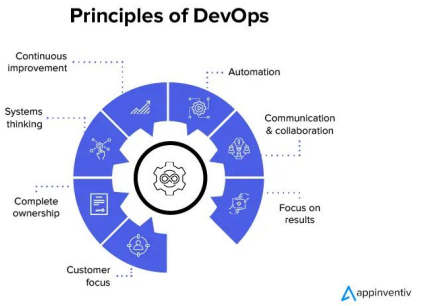
5. Monitoring and Logging Tools

- Prometheus: An open-source monitoring system that collects and analyzes system performance data.
- Grafana: A tool for visualizing monitoring data from Prometheus.
- ELK Stack (Elasticsearch, Logstash, Kibana): A system for managing and analyzing logs.

Devops Toolchain

Typical DevOps Toolchain Process

- Step 1: Source code is stored on Git (GitHub, GitLab).
- Step 2: Whenever changes are made to the source code, Jenkins (or another CI tool) triggers the build, testing, and integration process.
- Step 3: If the tests pass, the source code is deployed to a containerized environment (using Docker).
- Step 4: Kubernetes manages the deployment and scaling of the application.
- Step 5: The system is continuously monitored through Prometheus and Grafana, and logs are collected through the ELK Stack.



DevOps Principles

DevOps Principles

---

1. Continuous Collaboration
2. Automation
3. Continuous Integration - CI
4. Continuous Delivery - CD
5. Continuous Monitoring and Feedback

Devops Best Practices

Devops Best Practices

---

1. Best Practices for Continuous Integration (CI)
   - Integrate source code at least daily: Ensure code is integrated frequently to avoid conflicts between development branches.
   - Automate testing: Automatically test code upon integration to detect errors early.
   - Maintain a clean development environment: Ensure that the development environment is free of issues when integrating new code.

Example: Use Jenkins or GitLab CI to automate the source code integration and testing process.

Devops Best Practices

---

2. Best Practices for Continuous Deployment (CD)

- Deploy in increments: Deploy small, manageable releases to minimize risk.
- Monitor and test after deployment: Ensure that new releases do not cause errors and that the software remains stable after each deployment.
- Feature Flags: Use feature toggles to enable/disable new features without redeploying the entire system.

Example: Tools like CircleCI or Travis CI can help automate the continuous deployment process.

Devops Best Practices

---

3. Best Practices for Automated Testing

- Write tests from the beginning: Integrate testing into the development process from the moment you start writing code.
- Build comprehensive test suites: Include unit tests, integration tests, and system tests in the automated test suite.
- Ensure continuous execution of tests: Automate the testing process and run tests every time changes are made to the code.

Example: Tools like JUnit, Selenium, and TestNG are popular tools for automating tests.

Devops Best Practices

---

4. Best Practices for Infrastructure as Code (IaC)

- Code the entire infrastructure: All infrastructure and configurations should be coded to make management and modifications easier.
- Use automation tools like Terraform or Ansible: These tools help create, manage, and provision infrastructure automatically and efficiently.
- Manage infrastructure state: Monitor and store the state of the infrastructure to avoid reconfiguration and ensure consistency.

Example: Use Terraform or Pulumi to deploy infrastructure as code.

Devops Best Practices

---

5. Best Practices for Version Control

- Use a distributed version control system like Git: Git makes it easy to manage source code and control changes.
- Branching properly: Create separate development branches for each feature or bug fix to avoid code conflicts.
- Ensure clean code: Before merging into the main branch, code should be thoroughly reviewed and tested.

Example: Git, GitHub, and GitLab are popular version control tools.

Devops Best Practices

---

6. Best Practices for Monitoring and Logging

- Monitor system performance and health: Use monitoring tools to track the status of applications, servers, and services.
- Integrate logging and alerts: Error messages and alerts must be logged and analyzed to detect issues early.
- Analyze logs to optimize: Analyze logs to understand the root cause of issues and improve system performance.

Devops Best Practices

---

7. Best Practices for Continuous Improvement

- Continuous improvement involves always enhancing the software development and deployment processes using modern methods and tools.
- Integrate feedback from users and other teams: Use feedback from customers and teams to improve both the process and the product.
- Manage software versions and continuous updates: Regularly update the software and optimize the development process in each iteration.

Devops Best Practices

---

8. Improving Communication and Collaboration

One of the key factors in DevOps is enhancing collaboration and communication between development, testing, and operations teams.

- Hold regular meetings: Ensure that development and operations teams are aligned on progress and issues.
- Enhance communication through work management tools: Tools like JIRA, Slack, and Trello help track and address issues during the development process.

Essential DevOps Tools

Infrastructure as Code (IaC)

Use tools like Terraform, AWS CloudFormation, or Ansible to automate the management of infrastructure.

Essential DevOps Tools

Cloud Computing or On-premise Infrastructure

Knowledge of managing and deploying applications on cloud platforms such as AWS, Microsoft Azure, Google Cloud, or on-premise infrastructure like VMware.

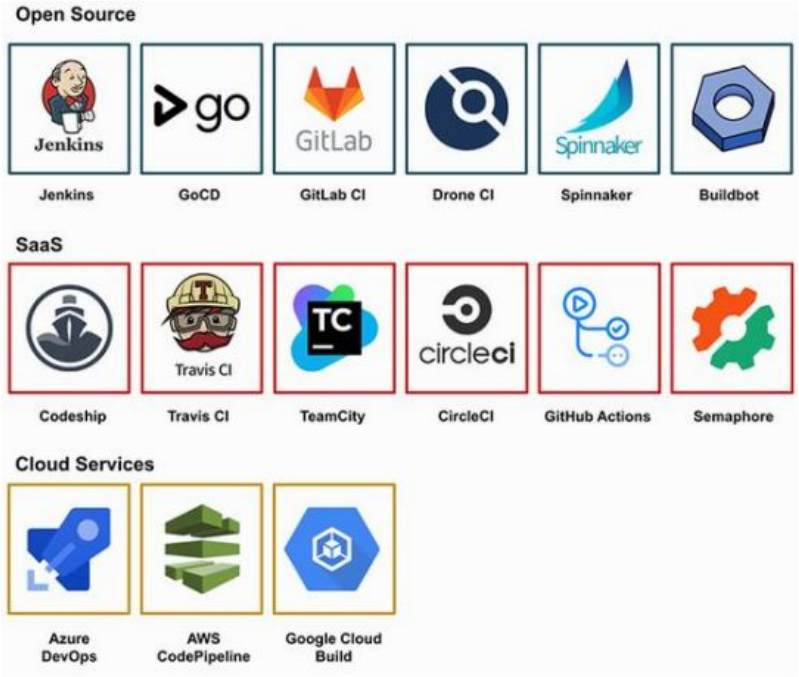
Essential DevOps Tools

Containerization: Deep understanding of Docker and container orchestration (management) tools like Kubernetes to manage and orchestrate containers in a production environment.

Essential DevOps Tools

Continuous Integration and Continuous Deployment (CI/CD)

- **CI/CD Tools**: Skills in working with tools like Jenkins, GitLab CI, and GitHub Actions to automate the build, test, and deployment processes.
- **Pipeline Development**: Skills in setting up and optimizing CI/CD pipelines to accelerate the development and deployment process of software.



Essential DevOps Tools

Configuration Management & Automation

- **Configuration Management Tools**: Using tools like Ansible to automate system configuration, ensuring consistency and ease of managing infrastructure.
- **Scripting**: Skills in writing scripts (Python, Bash, PowerShell, etc.) to automate system and application management tasks.

Essential DevOps Tools

Monitoring and Logging

- **Monitoring**: Knowledge of system and application monitoring tools such as Prometheus, Grafana, Nagios, and Datadog to track performance, set up alerts, and ensure system stability.
- **Logging**: Using tools like the ELK Stack (Elasticsearch, Logstash, Kibana), Fluentd to manage and analyze logs, helping to detect issues early in the system.

Essential DevOps Tools

Database Management

Database Knowledge (SQL and NoSQL): Understanding of databases such as MySQL, PostgreSQL, MongoDB, and Cassandra to support database management in the DevOps process.

Thank you